A Closer Look at Methods and Classes (Chapter 7 of Schilit)

Object Oriented Programming BS AI Section A , B, D

By Rashida

Revision of Methods/Functions

- What is a method/function?
- Parameter vs Argument
- Void?
    - Why do we use these methods

Polymorphism

OOP Principle

When an object of class shows multiple behaviors.

Overloading Methods

Two or more methods in same class:

Having same name

Same return type

Different type/number/Sequence of arguments

Type or number of argument determine the method to be called

Overloading Methods

- Return type alone is not sufficient during the call of method
  - Java matches the arguments with the parameters to call a particular method

Overloading Methods

```
// Demonstrate method overloading.
class OverloadDemo {
  void test() {
    System.out.println("No parameters");
  }

  // Overload test for one integer parameter.
  void test(int a) {
    System.out.println("a: " + a);
  }

  // Overload test for two integer parameters.
  void test(int a, int b) {
    System.out.println("a and b: " + a + " " + b);
  }

  // Overload test for a double parameter
  double test(double a) {
    System.out.println("double a: " + a);
    return a*a;
  }
}
```

What should be the output now???

```
class Overload {
  public static void main(String args[]) {
    OverloadDemo ob = new OverloadDemo();
    double result;

    // call all versions of test()
    ob.test();
    ob.test(10);
    ob.test(10, 20);
    result = ob.test(123.25);
    System.out.println("Result of ob.test(123.25): " + result);
  }
}
```

Example of Automatic Type conversion

```
// Automatic type conversions apply to overloading.
class OverloadDemo {
  void test() {
    System.out.println("No parameters");
  }

  // Overload test for two integer parameters.
  void test(int a, int b) {
    System.out.println("a and b: " + a + " " + b);
  }

  // Overload test for a double parameter
  void test(double a) {
    System.out.println("Inside test(double) a: " + a);
  }
}
```

Example of Automatic Type conversion

```
class Overload {
  public static void main(String args[]) {
    OverloadDemo ob = new OverloadDemo();
    int i = 88;

    ob.test();
    ob.test(10, 20);

    ob.test(i); // this will invoke test(double)
    ob.test(123.2); // this will invoke test(double)
  }
}
```

Overloading Methods

Advantage is common name methods for different activities of same nature, exp: Addition

Those languages which don’t support overloading(like C):

- Give unique names for methods
- abs() for int and labs() for long int

It is one of the ways in which java can achieve polymorphism:

- One object shows multiple behaviors

Overloading Constructors

- Requires three parameters

```
class Box {
  double width;
  double height;
  double depth;

  // This is the constructor for Box.
  Box(double w, double h, double d) {
    width = w;
    height = h;
    depth = d;
  }

  // compute and return volume
  double volume() {
    return width * height * depth;
  }
}
```

```
Box ob = new Box();
```

Overloading Constructors

- This is invalid right now

- What if you wanted a box without parameters

Some Questions?

What if you wanted a Box and didn’t care about initial dimensions?

What if you wanted to Initialize a cube by specifying only one value that will be used for all other dimensions

```
/* Here, Box defines three constructors to initialize
   the dimensions of a box various ways.
*/
class Box {
  double width;
  double height;
  double depth;

  // constructor used when all dimensions specified
  Box(double w, double h, double d) {
    width = w;
    height = h;
    depth = d;
  }

  // constructor used when no dimensions specified
  Box() {
    width = -1;  // use -1 to indicate
    height = -1; // an uninitialized
    depth = -1;  // box
  }

  // constructor used when cube is created
  Box(double len) {
    width = height = depth = len;
  }

  // compute and return volume
  double volume() {
    return width * height * depth;
  }
}
```

```
class OverloadCons {
  public static void main(String args[]) {
    // create boxes using the various constructors
    Box mybox1 = new Box(10, 20, 15);
    Box mybox2 = new Box();
    Box mycube = new Box(7);

    double vol;

    // get volume of first box
    vol = mybox1.volume();
    System.out.println("Volume of mybox1 is " + vol);

    // get volume of second box
    vol = mybox2.volume();
    System.out.println("Volume of mybox2 is " + vol);

    // get volume of cube
    vol = mycube.volume();
    System.out.println("Volume of mycube is " + vol);
  }
}
```

Passing Objects as Parameters to Methods

```
// Objects may be passed to methods.
class Test {
  int a, b;

  Test(int i, int j) {
    a = i;
    b = j;
  }

  // return true if o is equal to the invoking object
  boolean equalTo(Test o) {
    if(o.a == a && o.b == b) return true;
    else return false;
  }
}

class PassOb {
  public static void main(String args[]) {
    Test ob1 = new Test(100, 22);
    Test ob2 = new Test(100, 22);
    Test ob3 = new Test(-1, -1);

    System.out.println("ob1 == ob2: " + ob1.equalTo(ob2));
    System.out.println("ob1 == ob3: " + ob1.equalTo(ob3));
  }
}
```

Also used to copy values of one object to other with the help of constructors

```
// Here, Box allows one object to initialize another.

class Box {
  double width;
  double height;
  double depth;

  // Notice this constructor. It takes an object of type Box.
  Box(Box ob) { // pass object to constructor
    width = ob.width;
    height = ob.height;
    depth = ob.depth;
  }
```

A closer look at Argument passing

- Pass by value
- Pass by reference
- Parameter vs Argument

Pass By Value

- If the parameters are primitive types:
  - They are passed by value always in java
  - A copy of those values is given to the method
  - What every operation is performed inside the method over those values, doesn’t affect the original value (Copy is passed)

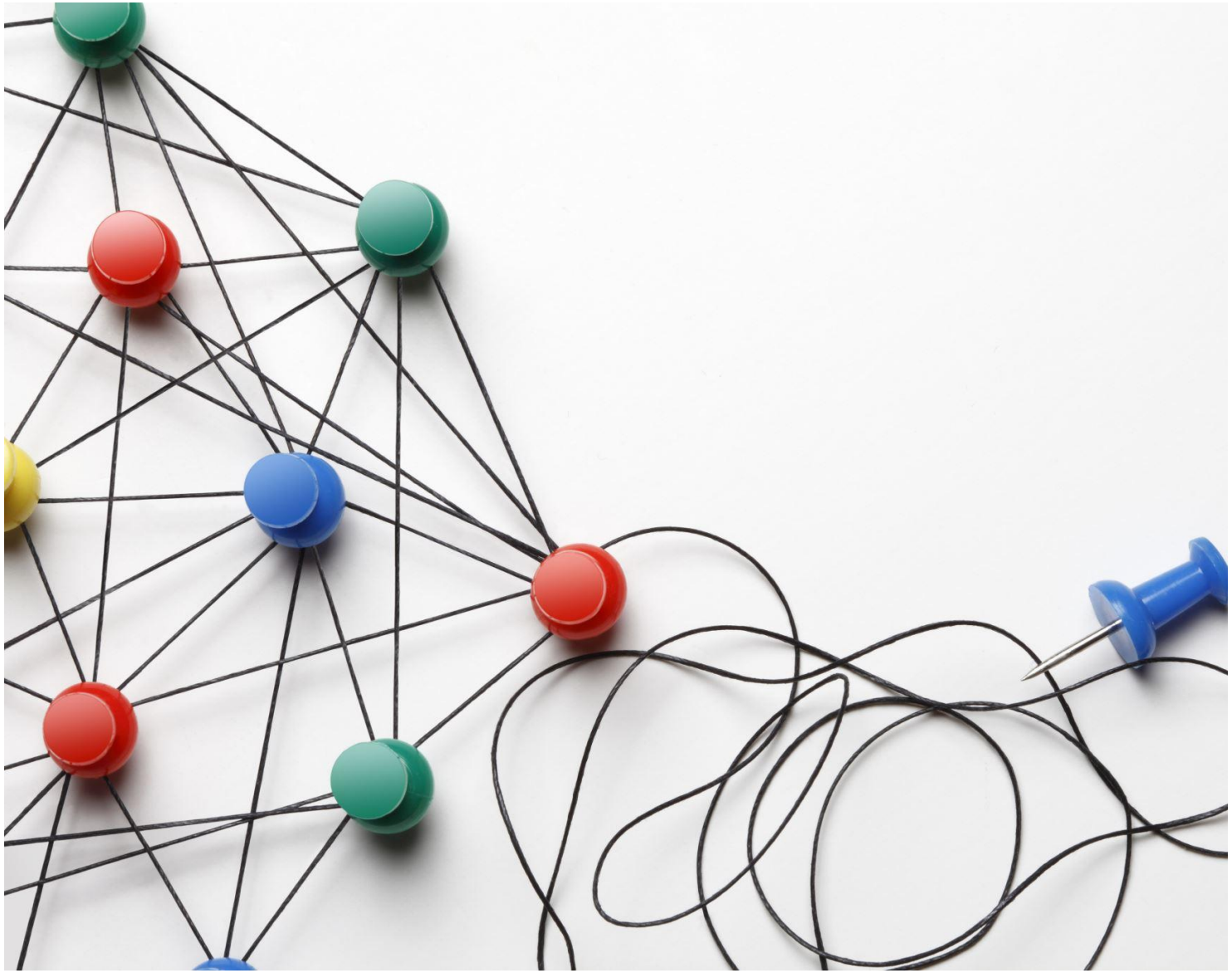
Demo

Pass By Reference

- If the parameters are Object types:
  - They are passed by reference in java
  - Because you pass a reference to memory location
  - Changes made through reference are affected at actual location

Demo

Returning objects from a method as return type



Recursion [not covered]

- A process:
  - Method calls itself
    - Recursive Method

Demo

Encapsulation

Process of wrapping up code and data in a single unit

Exp: Capsule mixed from several herbs

We can create a fully encapsulated class in Java by making all the data members of the class private.

How to access and modify the values then?

Getter and Setter Methods

Setter:	Getter:
• Assign/Modify the values of instance variables • Also known as Mutator • Return type is void	• Used to retrieve or get the values of instance variables • Also known as accessor • Return type is similar to the datatype of instance variable

Why getter/setter, when we have constructor

Constructor just for Initialization

Setter for initializing, as well as modifying/changing the values

Getter for retrieving

Advantages of Encapsulation

Class can be read-only/write-only

It provides you the **control over the data:**

Value of Marks should be greater than zero

Salary shouldn’t be negative

Data Hiding is also achieved, because private member can only be accessed inside the methods of same class

Code becomes more easy to understand and readable

Can we make constructor private?

- Yes you can but on one cost:
  - You can not create the object outside of the class

Demo

Task

- Create a class Student, a Student has Student_id, Student_name, and Program of study (Like CS,BBA etc), Make Five Objects of Student class
- Make Sure this class is a Encapsulated Class

```
public int i;
private double j;

private int myMethod(int a, char b) { //...
```

Access Controls in Java

- Access Modifiers:
  - How a member(instance variable/method) can be accessed
    - Public
    - Private
    - Protected (Applies only when Inheritance is involved)
    - Default

Access Modifiers

Private: The access level of a private modifier is only within the class. It cannot be accessed from outside the class.

Default: The access level of a default modifier is only within the package. It cannot be accessed from outside the package. If you do not specify any access level, it will be the default.

Protected: The access level of a protected modifier is within the package and outside the package through child class. If you do not make the child class, it cannot be accessed from outside the package.

Public: The access level of a public modifier is everywhere. It can be accessed from within the class, outside the class, within the package and outside the package.

Access Modifier	within class	within package	outside package by subclass only	outside package
Private	Y	N	N	N
Default	Y	Y	N	N
Protected	Y	Y	Y	N
Public	Y	Y	Y	Y

Group Discussion

Describe the use of improper access modifiers (all four). What kind of design and security issues does it lead?

When to use public?

When to use private rather than public?

When to use protected rather than private?

When to use default?

Give example of each.